

Acceptance Safety: A Conjunctive Release Framework for LLM-Generated Guard Specifications

Ye Xujun

East China Normal University
Shanghai, China

Abstract—Large language models synthesise code that often looks correct under everyday unit tests yet still mishandles thin specification regions—especially catch-all residuals and ordered overlaps. Consider a tier classifier whose OTHERWISE branch must return Review: an LLM candidate can pass every Reject/Pass sample while silently promoting the mid-band to Pass. A release dashboard that tracks mean pass rate alone will ship that defect.

We reframe the goal as acceptance safety: release only when every specification-derived witness matches *and* a structural screen clears ($\text{Accept} \Leftrightarrow \text{Conf}=1 \wedge \text{Screen}=\text{pass}$). The vehicle is a conjunctive release decision framework realised by HSP-Agile on SOFL/FSF: first-match SMT witnesses, a typed Semantic Feedback IR for repair under headroom, and a pluggable screen. Here FAR is the fraction of faulty mutants that a mode incorrectly releases; PDR is the complementary rejection rate.

When repair still has headroom, structured feedback rescues catch-all failures that unstructured dumps miss (E6: semantic_ir 86.9% vs. test_only 79.1%, +7.7 pp; W/L/T 14/4/102; 95% CI [2.3, 13.6] pp; $p=0.018$; all 14 Conf. wins are others rescues, none are ordering repairs). At comparable mean conformance, conjunctive Accept cuts escaped defects (E2: M FAR 5.0% vs. B2 8.8%; PDR 95.0% vs. 91.2%). At matched budget $K=3$, equal- K Conf. places M_eq at 100.0% vs. B2 97.5% (+2.5 pp; 3/0/117). Prefer test-feedback (B2) by default; escalate to full mode M when release requires $\text{Accept}=1$ or $\text{FAR} \leq 5\%$ and headroom remains.

Index Terms—acceptance safety, conjunctive release, specification-guided repair, ordered guards, LLM synthesis, SOFL, SMT, false accept rate

I. INTRODUCTION

A release dashboard can look healthy while a residual specification region is already broken. LLMs synthesise code that passes busy unit tests yet mishandles catch-all OTHERWISE bands and ordered overlaps that everyday suites never sample [1]–[3]. SOFL/FSF makes those regions explicit as first-match scenarios [4]–[6]. We ask whether a team can *refuse* to ship under an acceptance-safety bar that uses the specification itself as evidence—without full deductive verification.

Acceptance safety. Release requires every specification-derived witness to match *and* a structural screen to clear: $\text{Accept} \Leftrightarrow \text{Conf}=1 \wedge \text{Screen}=\text{pass}$. We call this rule a **conjunctive release decision framework**. HSP-Agile realises it on SOFL/FSF: SpecIR witnesses, typed Semantic Feedback IR under headroom, and a pluggable screen. FAR (False Accept Rate) is the fraction of faulty mutants that a mode incorrectly releases; PDR is the complementary rejection rate.

Primary vignette (catch-all). A tier classifier rejects negative scores and passes scores at or above a floor; everything else falls into OTHERWISE and must return Review. An LLM candidate collapses that residual to Pass. Busy Reject/Pass tests succeed; the mid-band never reaches a human. Typed IR names the residual scenario (others)—the same regime as all 14 E6 Conf. wins over test-only feedback (0/14 ordering repairs). The intro uses a three-scenario skeleton with residual S_3 ; BENCH-120 hard tasks scale the same first-match shape to eight scenarios, so coded wins refer to residual index S_8 .

Secondary vignette (ordering). Ordered tax/sensor overlaps motivate first-match witnesses Φ_i and prevention mutants that reorder or drop preemption; they are not the E6 win set. A public API ACL reconstruction (enforce_api_acl) fail-opens on OTHERWISE while guest/admin tests pass—Accept stays red.

Contributions.

- **C3—Conjunctive release (lead):** Accept improves FAR/PDR vs. mean Conf. alone (E2: M FAR 5.0% vs. B2 8.8%; PDR 95.0% vs. 91.2%).
- **C2—Typed feedback under headroom:** +7.7 pp vs. test-only at matched $K=3$ (E6; 14/4/102; CI excludes 0); all 14 wins are catch-all/others rescues.
- **C4—Deployment:** Prefer B2 by default; escalate to M when $\text{Accept}=1$ or $\text{FAR} \leq 5\%$ and headroom remains; ranking pilots often saturate under B2.

Causal chain: Accept (E2) as the release predicate; E6 as the repair mechanism when headroom remains; equal- K and external pilots as C4 guidance. Near-ceiling ranking runs (E1/E12) provide saturation context only. A companion long report carries full ablations; this cut keeps the evidence needed for C2–C4.

II. RELATED WORK

Closest LLM+verifier systems differ on whether witnesses respect first-match order, whether repair carries scenario structure, and whether prevention (PDR/FAR) is measured (Table I).

Proof-oriented oracles. DafnyBench and Clover target proof discharge [7], [8]; success is a theorem, not first-match re-

TABLE I
POSITIONING (TICK = DESIGN GOAL).

Approach	Ordered Φ_i	Typed IR	Prev.	Gate
Clover / DafnyBench	—	partial	—	proof
Self-Debug / Reflexion	—	—	—	—
ChatRepair / APR	—	—	—	tests
B2 test-feedback	—	—	—	tests
B6 VerifierLoop-FSF	✓	—	—	cex
SgDP / M	✓	✓	✓	Accept

gion coverage. Our Conf. metric is witness agreement under SpecIR, not a proof certificate. CEGIS-style SMT loops are closer in spirit, but typically synthesise programs or invariants from logical specs rather than ordered-guard FSF with typed repair records.

Verbal and agentic repair. Self-Refine, Self-Debug, and Reflexion raise generation success via critique or verbal memory [9]–[11]. That channel is LLM-as-judge style: useful, but not bound to an executable first-match oracle—the judge may omit the catch-all scenario index. CodeT and coding agents optimise repository tasks without an FSF oracle [12]–[16]. LLM-based APR patches failing tests [17]–[21]; we evaluate unstructured test feedback as B2 and isolate typed IR in E6.

Adjacent notations and SOFL. Alloy and TLA+ encode structured constraints [22], [23]; SpecIR adapters are the intended bridge when guards are first-match and SMT-encodable. Contracts and proof-oriented LLM loops occupy complementary points [24], [25]. SOFL has a long industrial and teaching lineage [4]–[6]; railway and crossing cases motivate ordered-guard contracts but do not release public .asf1 dumps [26], [27]. Related SOFL fault-prevention work studies specification-to-program defects [28]; we target LLM repair under an executable SpecIR oracle. SgDP combines first-match SMT witnesses, typed Semantic Feedback IR, and conjunctive Accept under PDR/FAR.

III. APPROACH

A. SpecIR and first-match witnesses

For task t with ordered scenarios $\langle s_1, \dots, s_n \rangle$, SpecIR exposes first-match regions as a shared hinge across notations (SOFL/FSF, Mini-Z, Mini-StateMachine adapters). Each scenario $s_i = (G_i, P_i)$ pairs a guard with a postcondition; FSF selects the least i with $G_i(\mathbf{x})$ and requires $P_i(\mathbf{x}, \mathbf{y})$. For scenario s_i , the witness generator solves

$$\Phi_i \triangleq \hat{g}_i(\mathbf{x}) \wedge \bigwedge_{j < i} \neg \hat{g}_j(\mathbf{x}) \quad (1)$$

with Z3 [29], [30], yielding suite $\mathcal{D}(t)$. Without the negated higher guards, a model of g_i alone may still belong to s_j for $j < i$. An *others* witness uses $\bigwedge_i \neg \hat{g}_i$ when nonempty; a prior *others*-witness bug (*others* encoded without $\neg$ prior guards) inflated false failures and is disclosed as a measurement correction—we cite fixed-oracle runs as primary. Soundness is conditional on a faithful SMT encoding of SpecIR guards

(integer/boolean fragments in our corpora). Random tests undersample Φ_i as overlap grows, so residual and ordering bugs survive green CI [3], [31].

B. Catch-all repair step

Return to the tier classifier: on a mid-band input the buggy candidate returns Pass where the oracle requires Review. The Semantic Feedback IR records (abbreviated):

```
{
  "violation_type": "others",
  "scenario_index": 2,
  "constraint_text": "OTHERWISE residual",
  "expected": "Review",
  "observed": "Pass",
  "suggested_fix": "return Review on catch-all"
}
```

FeedbackRenderer projects this JSON into the repair prompt. E6 isolates `test_only` / `test_expected` / `semantic_ir` surfaces under identical K . (A secondary ordering example uses inputs such as $(-3, -10)$ with `violation_type:ordering`; that trail motivates Φ_i and E2 mutants, not the E6 win profile.)

C. Semantic Feedback IR and Accept

The typed IR carries at least: `violation_type` (ordering, postcondition, others), `scenario_index`, `constraint_text`, concrete inputs, `expected`/observed outputs, and an optional `suggested_fix` hint. Conformance and release:

$$\text{Conf}(c, t) = \frac{1}{|\mathcal{D}(t)|} \sum_{\mathbf{x} \in \mathcal{D}(t)} \mathbf{1}[f_c(\mathbf{x}) = g_t(\mathbf{x})], \quad (2)$$

$$\text{Accept}(c, t) \iff \text{Conf}(c, t) = 1 \wedge P^H(c) = 0. \quad (3)$$

Mode B2 uses unstructured test pass/fail only—no scenario index, no constraint text. Controlled E6 isolates feedback content under full M at matched $K=3$: variant A is test-only; B adds expected outputs; C is the full Semantic IR. Equal- K mode M_{eq} ($K=3$, `semantic_ir`, advisory gate) is the matched-budget Conf. ranking vs. B2. Strengthened E1 ($K=5$, `execution_trace_matched`, advisory gate) is a multi-factor bundle used as a near-ceiling ranking check. E14 further shows that length-matched execution traces can outperform `semantic_ir` on mean Conf. (85.4% vs. 75.1%); the E6 claim is therefore relative to unstructured test feedback.

D. Pipeline

Algorithm 1 summarises one task. Synthesise from FSF text $\rightarrow$ check on $\mathcal{D}(t) \rightarrow$ if not Accept, render Feedback IR and repair, up to budget K ; on exhaustion return $\arg \max_k \text{Conf}(c_k, t)$. Prevention evaluation injects specification-confusion and implementation-screening mutants and scores PDR/FAR on the faulty set. High-severity pattern screen P^H blocks known dangerous templates (e.g., inverted priority, missing fault preemption) even when mean Conf. is high. Overlap density is an engineering hinge for witness generation: E3/E10 tertiles are non-monotone, so overlap alone is not a deployment trigger. The deployable levers are feedback typing (E6) and the Accept/prevention gate (E2).

Algorithm 1 SgDP repair loop (HSP-Agile instantiation)

Require: FSF task t , budget K , feedback mode m

- 1: $\mathcal{D}(t) \leftarrow \text{SpecIR witnesses via Z3}$
- 2: $c^* \leftarrow \perp$; $best \leftarrow 0$
- 3: **for** $k = 1$ to K **do**
- 4: $c \leftarrow \text{synthesise / repair}(t, \text{feedback}_{k-1})$
- 5: $conf \leftarrow \text{Conf}(c, t)$ on $\mathcal{D}(t)$
- 6: **if** $conf > best$ **then** $c^* \leftarrow c$; $best \leftarrow conf$
- 7: **end if**
- 8: **if** $\text{Accept}(c, t)$ **then return** c
- 9: **end if**
- 10: $\text{feedback}_k \leftarrow \text{render}(m, \text{failures on } \mathcal{D}(t))$
- 11: **end for**
- 12: **return** c^* ▷ best-effort argmax Conf.

IV. EXPERIMENTAL SETUP

Benchmarks. **BENCH-120:** synthetic hard FSF (5 inputs, 8 scenarios) retained by a Z3 overlap filter; primary corpus for E1/E6/E12. **E10:** removes the filter ($n=100$) to bound filter bias. **Industrial-pattern** ($n=31$): practitioner-style decision tables. **Desensitized published-industrial** ($n=28$): FSF reconstructed from published railway/ATM/banking SOFL cases [5], [26], [27]. **GitHub harvest** ($n=48$) and **HKCA09**→FSF ($n=35$) bound public transfer for C4. Vendor .asf1 dumps are not redistributed; an import path exists for future NDA pilots (Table II).

Modes and metrics. B1 one-shot; B2 test-feedback ($K=3$); M (strengthened bundle, $K=5$); M_eq (equal- K , $K=3$, semantic_ir, advisory). E6 variants A/B/C hold full M fixed and vary only feedback surface. Metrics: mean Conf.; Strict / Accept; PDR/FAR on mutants; paired wins/losses/ties with bootstrap CIs where claimed. Primary LLM: ecnu-plus (Qwen-family); secondary endpoints bound model dependence. Repair temperature $T_{\text{repair}}=0$; witness budget up to 32 formal cases per task in strengthened runs.

Mutants (E2). Implementation-screening mutants flip guard order, drop fault/conflict preemption, or weaken postconditions ($n=852$; pooled with specification-confusion yields $n=1704$). PDR is the fraction of faulty candidates rejected; FAR is the fraction of faulty mutants incorrectly released under the release predicate.

Protocol. Cite E6 for C2; M_eq for equal- K Conf. ranking; strengthened E1 as near-ceiling context. Authoritative run ids: `run_feedback_v2` (E6), `prevention_full_v1` (E2), `run_e1_equal_k_v1`, `run_e1_m_win_v2`, `run_desens_real_sofl_v1`.

V. RESULTS

A. RQ1: Typed Semantic Feedback IR (E6 / C2)

Figure 1 and Table IV hold K and tasks fixed under full M ($K=3$, $n=120$). Variant C (full Semantic IR) reaches 86.9% mean Conf.—**+7.7 pp** over test-only (A, 79.1%) and +8.0 pp

TABLE II
CORPUS CARD (DO NOT MIX ROWS IN ONE CLAIM).

Corpus	n	Role
BENCH-120	120	Primary E1/E6/E12 (overlap-filtered)
E10 unfiltered	100	Filter-bias bound
Industrial-pattern	31	Practitioner-style tables (C4)
Desens. published	28	Reconstructed SOFL; all-mode 100% (C4)
GitHub harvest	48	Public .asf1; B2=M_eq (C4)
HKCA09→FSF	35	Public SOFL modules; B2=M_eq (C4)

TABLE III
FIELD STRUCTURE UNDER HARD COMBO SEEDS (GEMINI; $n=120$ CELLS).

FULL - X	Δ (pp)	W/L/T	Excl. 0
test_only	+33.2	47/8/65	Yes
test_expected	+32.6	47/8/65	Yes
ir_nl_only	+28.0	46/12/62	Yes
ir_no_scenario_id	-0.9	22/17/81	No
ir_no_expected	+5.0	29/17/74	No

over test+expected (B, 78.9%). Paired: **14/4/102**; 95% CI [2.3, 13.6] pp; Wilcoxon $p=0.018$. Most tasks tie near ceiling; the 14 wins are a rescue regime: 13/14 have test-only Conf=0, and IR records name catch-all scenario others with arithmetic/output mismatches (Table V). The four losses are mixed output/arithmetic residuals. Ordering/preemption defects still motivate Φ_i witnesses and E2 mutants. E14: semantic_ir 75.1% vs. execution_trace_matched 85.4% (5/19/96)—E6 is relative to unstructured feedback. Hard combo seeds on gemini-2.5-flash (pooled $n=80$): FULL vs. test-only +26.5 pp (CI excl. 0). Field ablation (Table III): FULL beats NL-only dumps of the same facts (+28.0 pp; CI excl. 0), while dropping any single IR field yields CIs that include 0—structure vs. unstructured text, not one-field necessity.

B. RQ2: Prevention (E2 / C3)

On implementation-screening mutants ($n=852$), M reaches PDR 95.0% / FAR 5.0% vs. B2 91.2% / 8.8%; paired bootstrap 95% CIs on the gaps exclude zero (Fig. 2). The absolute PDR lift is modest (+3.8 pp) but operationally meaningful at the FAR≤5% bar that many safety-adjacent release processes target. Fixed-oracle Conf. ablations vs. strengthened M: removing repair -25.8 pp; removing formal checker -17.5 pp; removing pattern guard 0.0 pp on Conf. under an advisory gate—repair and witnesses move Conf.; the pattern screen moves prevention (measure via E2).

C. RQ3: Equal- K ranking and near-ceiling context

Equal- K Conf. ranking (Table IV): M_eq 100.0% vs. B2 97.5% (+2.5 pp; 3/0/117)—matched budget, not a substitute for E6. Unified external equal- K : GitHub B2=M_eq 89.9%; HKCA09 both 100%; desensitized both 100%. Strengthened E1 ($K=5$ bundle, Table VI) sits near ceiling. E12: B2 reaches 100% on every seed; M mean 99.7% across seeds.

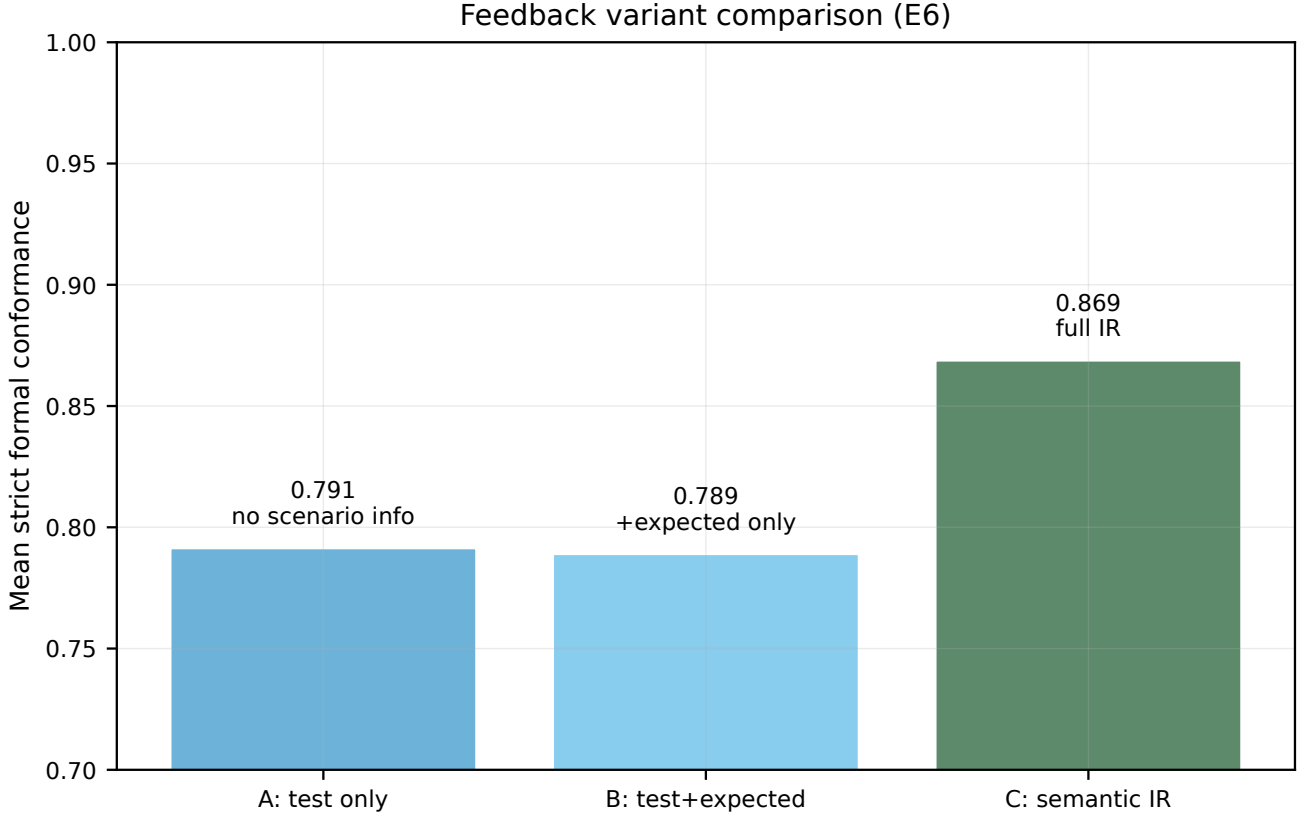


Fig. 1. E6 feedback variants under full M ($K=3$, $n=120$). Variant C (full Semantic IR) +7.7 pp vs. A; CI excludes 0.

TABLE IV
E6 PAIRED STATS, WIN PROFILE, AND EQUAL- K CONF. RANKING.

Contrast / mode	W/L/T or n	Conf. / Δ	95% CI
E6 C-A	14/4/102	+7.7 pp	[2.3, 13.6]
E6 C-B	14/4/102	+8.0 pp	[2.2, 14.0]
Win profile	13/14 zero-A	others/arith.	—
Equal- K B2	120	97.5%	—
Equal- K M_eq	120	100.0% (+2.5)	3/0/117

TABLE V
DEFECT-CLASS $\leftrightarrow$ EVIDENCE MAP.

Defect class	Primary evidence	Supports
Order / preemption	Φ_i ; E2 mutants	C3 gate
others/arith. rescue	E6 win coding	C2 vs. dumps
Saturated easy tasks	pilots ($n \geq 28$)	C4 default B2

TABLE VI
E1 NEAR-CEILING RANKING (FIXED ORACLE; $K=5$ BUNDLE FOR M).

Mode	Mean Conf.	Strict	Mean LLM calls
B1	84.2%	84.2%	1.00
B2	98.3%	98.3%	~ 1.18
M ($K=5$ bundle)	100.0%	100.0%	~ 1.16

TABLE VII
DEPLOYMENT BOUNDARY (C4): RELEASE BAR + HEADROOM.

Signal	Prefer B2	Prefer M
Default	Mean Conf./latency	—
Release	Mean Conf. OK	Accept=1
Prevention	—	FAR $\leq 5\%$
Headroom	Saturated endpoint	Collapsed test-only Conf. (E6)
Overlap tertile	Non-monotone	Not a trigger

D. RQ4: Deployment boundary (C4)

Canonical E3/E10 tertiles are non-monotone (Fig. 3): escalate on Accept/FAR and headroom, not on overlap density alone (Table VII). On fixed-oracle E1, mean LLM calls for B2 (~ 1.18) and strengthened M (~ 1.16) are comparable; wall-clock means are 9.6 s (B2) vs. 10.5 s (M).

E. External validity

On industrial-pattern SOFL ($n=31$), B2 reaches 100% Conf. under gpt-4o while B1/M sit at 95.5% (Table VIII). Desensitized published pilot ($n=28$): B1/B2/M_eq all 100.0%. GitHub harvest ($n=48$): B2/M_eq 89.9% vs. B1 85.8%. HKCA09 $\rightarrow$ FSF ($n=35$): B1 74.3% / B2=M_eq 100%. These ranking ties support C4 default-B2 on mean Conf. Where one-shot saturates, HKCA09 hard-seed repair (wrong_relop: semantic_ir 82.0% vs. test_only 69.3%, +12.7 pp; CI may include 0) is

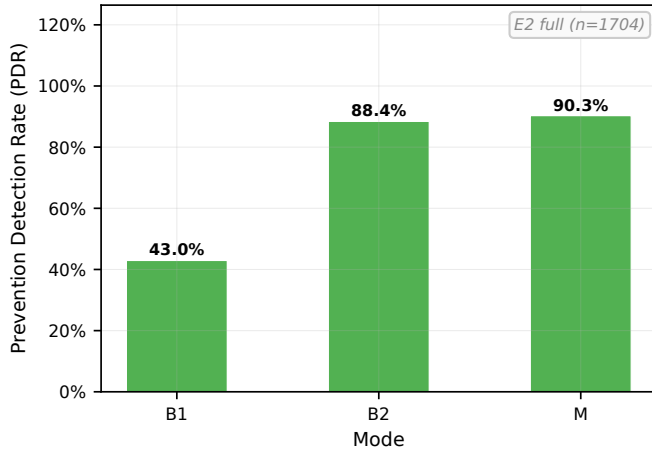


Fig. 2. E2 aggregate PDR by mode.

TABLE VIII
EXTERNAL CORPORA (MEAN CONF. %; C4).

Corpus	B1	B2	M / M_eq
Industrial-pattern ($n=31$, gpt-4o)	95.5	100.0	95.5
Desens. published ($n=28$)	100.0	100.0	100.0
GitHub harvest ($n=48$)	85.8	89.9	89.9
HKCA09 FSF ($n=35$)	74.3	100.0	100.0
Equal- K BENCH-120	—	97.5	100.0

scoped external support. Provenance: reconstructions / public harvest—not proprietary dumps.

F. Model dependence

On `ecnu-max`, E6 saturates ($\approx 89.2\%$ A/B/C); commercial endpoints show saturation, modest gain, or regression by model—typed-IR gains are headroom-dependent. Lead Conf. mechanism remains E6 under equal K on the primary LLM with repair headroom.

VI. DISCUSSION

Practice. Match feedback depth to the release bar: B2 for mean Conf. / latency; M when conjunctive Accept or $\text{FAR} \leq 5\%$ is required *and* repair headroom remains (E6 rescue when test-only Conf. collapses). External pilots reinforce default B2 on mean Conf. A practical playbook: (i) B2 inner loop; (ii) pre-release M with Accept; (iii) track FAR on a frozen mutant pack; (iv) escalate if FAR drifts or Accept failures cluster.

Limitations. BENCH-120 is synthetic and overlap-filtered; E10 and public pilots bound transfer. Finite witnesses approximate Conf.; the others-witness measurement correction previously inflated false failures. LLM non-determinism is mitigated by $T_{\text{repair}}=0$ and paired tests; the $K=5$ E1 bundle is not single-factor C2. Published reconstructions and harvests are not production dumps; vendor NDA `.asf1` pilots remain future work. Ceiling effects on stronger models shrink E6 deltas—report headroom when transferring claims. E14 shows that other structured traces can outperform Semantic IR on

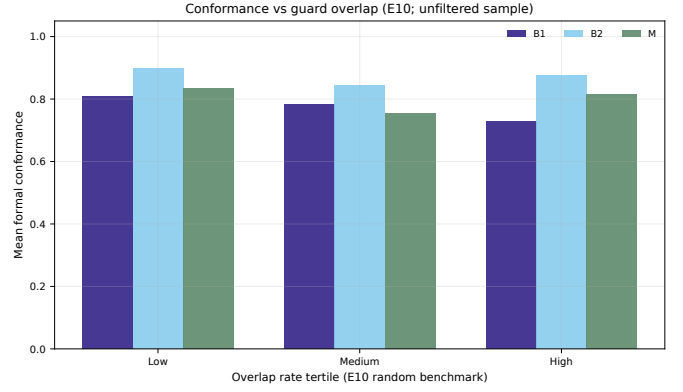


Fig. 3. E10 Conf. vs. overlap tertile (non-monotone).

mean Conf.; the uniqueness claim is scoped to unstructured test feedback. B2 leads every E12 seed (100%); M mean is 99.7% across seeds.

Reproducibility. Authoritative numbers and run ids ship with the artefact package (`AUTHORITATIVE_NUMBERS.md`, benchmark JSON, summarisation scripts). Primary claims cite those run directories rather than ad-hoc aggregates.

VII. CONCLUSION

SgDP/HSP-Agile pairs SpecIR witnesses, typed Semantic Feedback IR, and conjunctive Accept for ordered-guard specs. Acceptance safety is the lead story: release only under $\text{Accept} \Leftrightarrow \text{Conf}=1 \wedge \text{Screen}=\text{pass}$. Under headroom, E6 yields +7.7 pp vs. test-only at $K=3$ (CI excludes 0; 14/14 catch-all rescues). E2 justifies the gate at FAR 5.0% vs. 8.8%. Deploy B2 by default; escalate for Accept or $\text{FAR} \leq 5\%$ with headroom. Future work includes NDA vendor `.asf1` pilots, adaptive K , and SpecIR adapters beyond SOFL/FSF.

REFERENCES

- [1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, “Evaluating large language models trained on code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [2] A. M. Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M. C. Desmarais, and Z. M. Jiang, “GitHub Copilot AI pair programmer: Asset or liability?” *Journal of Systems and Software*, vol. 203, p. 111734, 2023.
- [3] H. Zhu, P. A. V. Hall, and J. H. R. May, “Software unit test coverage and adequacy,” *ACM Computing Surveys*, vol. 29, no. 4, pp. 366–427, 1997.
- [4] S. Liu, A. J. Offutt, C. Ho-Stuart, Y. Sun, and M. Ohba, “SOFL: A formal engineering methodology for industrial applications,” *IEEE Transactions on Software Engineering*, vol. 24, no. 1, pp. 24–45, 1998.
- [5] S. Liu, *Formal Engineering for Industrial Software Development: Using the SOFL Method*. Berlin, Heidelberg: Springer, 2004.
- [6] J. Li and S. Liu, “Requirements-related fault prevention mechanism for SOFL formal specification-based programming,” in *Proc. 22nd IEEE Int. Conf. Software Quality, Reliability and Security Companion (QRS-C)*. IEEE, 2022, pp. 359–367.
- [7] C. Loughridge, Q. Sun, S. Ahrenbach, F. Cassano, C. Sun, Y. Sheng, A. Mudide, M. R. H. Misu, N. Amin, and M. Tegmark, “Dafny-Bench: A benchmark for formal software verification,” *arXiv preprint arXiv:2406.08467*, 2024.

- [8] C. Sun, Y. Sheng, O. Padon, and C. Barrett, "Clover: Closed-loop verifiable code generation," in *Proc. 1st Int. Symp. AI Verification (SAIV 2024)*, ser. Lecture Notes in Computer Science, vol. 14846. Springer, 2024, pp. 134–155.
- [9] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang *et al.*, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [10] X. Chen, M. Lin, N. Schärli, and D. Zhou, "Teaching large language models to self-debug," in *International Conference on Learning Representations (ICLR)*, 2024.
- [11] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [12] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen, "CodeT: Code generation with generated tests," in *International Conference on Learning Representations (ICLR)*, 2023.
- [13] X. Wang, B. Li, Y. Song, F. F. Xu, X. Tang, M. Zhuge, J. Pan, Y. Song *et al.*, "OpenHands: An open platform for AI software developers as generalist agents," *arXiv preprint arXiv:2407.16741*, 2024, formerly known as OpenDevin.
- [14] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, "SWE-agent: Agent-computer interfaces enable automated software engineering," *arXiv preprint arXiv:2405.15793*, 2024.
- [15] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Demystifying LLM-based software engineering agents," in *Proceedings of the ACM on Software Engineering (PACMSE)*, vol. 2, no. FSE, 2025, pp. 801–824, agentless; arXiv:2407.01489.
- [16] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *International Conference on Learning Representations (ICLR)*, 2024.
- [17] C. S. Xia and L. Zhang, "Automated program repair via conversation: Fixing 162 out of 337 bugs for \$0.42 each using ChatGPT," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024, pp. 819–831, chatRepair; earlier arXiv title: Keep the Conversation Going.
- [18] N. Jiang, K. Liu, T. Lutellier, and L. Tan, "Impact of code language models on automated program repair," in *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 1430–1442.
- [19] C. Le Goues, M. Pradel, and A. Roychoudhury, "Automated program repair," *Communications of the ACM*, vol. 62, no. 12, pp. 56–65, 2019.
- [20] I. Bouzenia, P. Devanbu, and M. Pradel, "RepairAgent: An autonomous, LLM-based agent for program repair," *arXiv preprint arXiv:2403.17134*, 2024.
- [21] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, and S. H. Tan, "Automated repair of programs from large language models," in *Proceedings of the 45th International Conference on Software Engineering (ICSE)*, 2023, pp. 1469–1481.
- [22] D. Jackson, "Alloy: A lightweight object modelling notation," *ACM Transactions on Software Engineering and Methodology*, vol. 11, no. 2, pp. 256–290, 2002.
- [23] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Boston, MA: Addison-Wesley, 2002.
- [24] B. Meyer, *Object-Oriented Software Construction*, 2nd ed. Prentice Hall, 1997, design by Contract.
- [25] E. First, M. N. Rabe, T. Ringer, and Y. Brun, "Baldur: Whole-proof generation and repair with large language models," in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2023, pp. 1229–1241.
- [26] J. Luo, S. Liu, Y. Wang, and T. Zhou, "Applying SOFL to a railway interlocking system in industry," in *Structured Object-Oriented Formal Language and Method (SOFL+MSVL)*, ser. LNCS, vol. 10189. Springer, 2017, pp. 160–177.
- [27] S. Liu, M. Asuka, K. Komaya, and Y. Nakamura, "Applying SOFL to specify a railway crossing controller for industry," in *Proceedings of the 2nd IEEE Workshop on Industrial Strength Formal Specification Techniques*, 1998, pp. 16–27.
- [28] J. Li and S. Liu, "Requirements-related fault prevention during the transformation from formal specifications to programs," *IET Software*, vol. 17, no. 3, pp. 316–332, 2023.
- [29] L. de Moura and N. Bjørner, "Z3: An efficient SMT solver," in *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, ser. Lecture Notes in Computer Science, vol. 4963. Springer, 2008, pp. 337–340.
- [30] C. Barrett and C. Tinelli, "Satisfiability modulo theories," in *Handbook of Model Checking*, E. M. Clarke, T. A. Henzinger, H. Veith, and R. Bloem, Eds. Springer, 2018, pp. 305–343.
- [31] P. Ammann and J. Offutt, *Introduction to Software Testing*, 2nd ed. Cambridge University Press, 2016.